

FULL STACK COMMUNITY

BIZ HACK'26

ONE-DAY FULL STACK & SOFTWARE DEVELOPMENT HACKATHON

SOFTWARE REQUIREMENTS SPECIFICATION

Team Name	Nexus
Team ID	FSSD-023
Problem Statement No.	PS39
Problem Statement Title	Digital Parking Management System
Team Leader	Kavin S
Contact Number / Email	9042119252 / kavins.it24@bitsathy.ac.in

Team Members

#	Name	Role
1	Lokesh R	DataBase (BackEnd developer) / UIUX designer
2	Kavin S	BackEnd developer
3	Mithun RS	FrontEnd developer

01 PROBLEM UNDERSTANDING

In 3–5 lines, explain the problem in your own words. Who is affected, what breaks down today, and why does it matter?

Drivers waste significant time and fuel searching for open parking spaces, while parking facility operators lack real-time visibility and control over slot occupancy. Today, manual paper logs, uncoordinated spot selection, and the absence of live availability verification cause double-bookings, traffic congestion, and revenue leakage. This breakdown degrades driver experience and prevents administrators from efficiently monitoring operations, managing staff, or auditing entry/exit events.

02 PROPOSED SOLUTION

Describe what your team will build and how it solves the problem above. One short paragraph is enough — save the detail for Section 03.

Our team built ParkNow, a unified Digital Parking Management platform featuring a React Native mobile application for drivers and a React Web Dashboard for administrators, both synchronized in real-time via Supabase. The system verifies slot availability before confirming reservations, prevents double-booking, instantly updates slot statuses upon booking or release, and empowers administrators with full control over parking slots, staff rosters, reservation tracking, and live audit logs.

03 CORE FUNCTIONAL REQUIREMENTS

List only the features you plan to actually implement during the hackathon. Priority: High / Medium / Low.

Requirement / Feature	Priority
User Authentication & Role-Based Access Control (RBAC)	HIGH
Interactive Map & Geolocation Parking Search	HIGH
Real-Time Visual Slot Selection & Live Grid	HIGH
Booking Management & Digital QR Pass Generation	HIGH
Admin Staff Directory & Attendant Management	MEDIUM
Payment Gateway & Digital Receipt Processing	LOW

Real-Time Cross-Platform Event Push System

LOW

04 USER ROLES

List each actor/role in the system and what they can do.

User / Role	Main Responsibilities
Parking Customer (Client)	• Search nearby parking facilities on an interactive GPS map. • View live slot availability, pricing details, and slot maps. • Select specific parking slots and reserve entry/exit time windows. • Make digital payments and generate dynamic QR entry/exit passes.
Parking Lot Attendant (Staff)	• Scan driver QR passes using the built-in camera scanner at entry/exit gates. • Perform manual vehicle check-in/out via License Plate Number (LPN). • Allocate available parking slots manually for walk-in drivers.
Parking Administrator (Admin)	• Monitor live executive dashboards (total revenue, occupancy %, active traffic logs). • Manage parking lot layout, slot statuses (Available, Maintenance, VIP), and zone configs. • Provision and manage parking staff accounts and gate access permissions. • Audit complete payment logs, verification history, and system activity metrics.

05 SYSTEM WORKFLOW

Show the main flow of the application — a simple flowchart or numbered sequence of screens/actions is recommended over prose.

Customer:

Login → Find Parking → Select Slot → Book & Pay → Get QR → Navigate → Entry → Park

Staff:

Login → Scan QR / LPN → Verify Booking → Gate Entry → Occupied → Exit → Payment → Slot Available

Admin:

Login → Dashboard → Manage Slots → Manage Staff & Gates → Monitor Bookings

06 TECHNOLOGY STACK

Default MERN layout shown below — edit any row to match your actual stack.

Layer	Technology / Tool
Frontend	React.js
Backend	Node.js + Express.js
Database	MongoDB
Authentication	JWT / bcrypt
Hosting / Deployment	Vercel
Other Tools	React Native for Application

07 DATABASE / DATA REQUIREMENTS

List the main entities/collections and the key fields each one stores.

Entity / Table	Key Data / Fields
users	user_id, full_name, email, phone, password_hash, role_id, created_at
parking_locations	location_id, name, address, latitude, longitude, hourly_rate, total_capacity, operating_hours
parking_slots	slot_id, location_id, slot_number, floor_level, status (AVAILABLE, RESERVED, OCCUPIED, MAINTENANCE)
vehicles	vehicle_id, user_id, vehicle_number (License Plate), vehicle_type (Car/Bike/EV), model_name

08 NON-FUNCTIONAL REQUIREMENTS

Brief expectations around quality attributes — keep each to one line.

Category	Expectation
Performance	App screens load within 1.5 seconds and real-time slot state changes sync across devices in under 500ms.
Security	Passwords hashed using bcrypt/Supabase Auth with mandatory JWT token validation and Role-Based Access Control (RBAC) on all APIs.

Usability	Mobile-responsive touch layout allowing drivers to discover and complete slot reservations in under 4 taps.
Availability	App services and database maintain 99.9% continuous uptime throughout the hackathon evaluation window.

09 MVP SCOPE

Separate what's essential to demo from what's a stretch goal if time allows.

Must-Have Features – Core MVP for Demo

Multi-Role Authentication & Workspace Switcher
(Driver, Staff, Admin)

Interactive Map Search with Nearby Parking & Live
Slot Counts

Visual Slot Selection Grid with Live Availability

Booking Reservation & Dynamic QR Pass Generation

Staff QR Scanner for Entry/Exit Verification

Manual Check-in/out using License Plate Number
(LPN) & Cash Collection

Optional – If Time Permits / Stretch Goals

Turn-by-Turn GPS Navigation

Live Payment Gateway Sandbox (Stripe/Razorpay)

Automated License Plate Recognition (ALPR)

Push Notifications

Multi-Language Support

CSV / PDF Report Export

10 EXPECTED OUTPUT

In 2–3 lines, describe what the final working demo should show.

The final working demo will showcase an end-to-end smart parking ecosystem: a driver searches for nearby facilities, reserves a specific slot, and receives a dynamic QR booking pass; a parking attendant scans the QR code at the gate to grant entry, triggering sub-second WebSocket updates that reflect the newly occupied spot instantly across the Admin analytics dashboard and all connected client apps

TEAM CONFIRMATION

We confirm that this document represents our proposed implementation for the selected BIZ HACK'26 problem statement.

Team Leader Signature: **Kavin s**

Date: 23.09.2026